



ElastiCache - In-Memory Database



ELASTICACHE (IN-MEMORY DATABASE)



1. TỔNG QUAN

- └ Purpose: Managed Redis or Memcached
- └ In-memory **database** (high performance, low latency)
- └ Reduce load on **databases** (read-intensive workloads)
- └ Make application **stateless** (session store)
- └ **AWS manages**: OS maintenance, patching, setup, monitoring, failure recovery
- └ ⚠ Requires heavy application code changes

Ưu điểm của ElastiCache:

- **Microsecond latency** - Cực kỳ nhanh so với database thông thường
- **Giảm tải cho database** - Cache các queries thường xuyên
- **Stateless application** - Lưu session data để app có thể scale horizontally
- **Managed service** - AWS lo toàn bộ infrastructure

Lưu ý:

- Multi-AZ support
- Read replica cho HA (Redis only)
- Data durability using AOF (Redis only)



2. REDIS vs MEMCACHED

Redis - High Availability & Persistence

REDIS	
✓ Multi-AZ with Auto-Failover	
✓ Read Replicas (scale reads + high availability)	
✓ Data Durability using AOF persistence	
✓ Backup and restore features	
✓ Supports Sets and Sorted Sets	
✓ Use case: HA, persistence, complex data structures	

Redis Features:

- **Multi-AZ** với automatic failover
- **Read Replicas** để scale reads và tăng HA
- **Data Durability** sử dụng AOF (Append-Only File) persistence
- **Backup & Restore** với retention up to 35 days
- **Advanced Data Structures:** Sets, Sorted Sets, Lists, Hashes
- **Use case:** Gaming leaderboards, real-time analytics, session store

Memcached - Simple & Fast Cache

MEMCACHED	
✓ Multi-node for partitioning (sharding)	
✗ No high availability (replication)	

- | | |
|---|--|
| ✗ Non persistent | |
| ✗ No backup and restore | |
| ✓ Multi-threaded architecture | |
| ✓ Use case : Pure cache, simple data | |

Memcached Features:

- **Multi-node** để partitioning (sharding) data
- **Multi-threaded** architecture
- **Không có HA** - No replication
- **Không persistent** - Chỉ là pure cache
- **Không có backup/restore**
- **Use case**: Simple caching layer, distributed cache

So sánh tổng hợp

Chọn Redis khi:

- Cần High Availability (Multi-AZ + Read Replicas)
- Cần data persistence và backup/restore
- Sử dụng complex data structures (Sets, Sorted Sets)
- Cần advanced features như pub/sub, transactions

Chọn Memcached khi:

- Chỉ cần simple key-value cache
- Cần multi-threaded performance
- Không cần persistence hay HA
- Cần horizontal scaling thông qua sharding

3. ELASTICACHE ARCHITECTURE PATTERNS

Pattern 1: Lazy Loading (Cache-Aside)

- └─ Pattern 1: Lazy Loading (Cache-Aside)
 - └─ App queries cache first
 - └─ Cache miss → Read from DB → Write to cache
 - └─ Pro: Only requested data is cached
 - └─ Con: Data can become stale

Flow:

1. Application query cache trước
2. **Cache hit:** Return data từ cache (fast)
3. **Cache miss:**
 - Read từ database
 - Write vào cache
 - Return data

Ưu điểm:

- Chỉ cache dữ liệu thực sự được sử dụng
- Đơn giản, dễ implement

Nhược điểm:

- Cache miss penalty (phải đợi query DB)
- Data có thể bị stale nếu không update

Pattern 2: Write Through

- └─ Pattern 2: Write Through
 - └─ Write to DB → Write to cache (sync)
 - └─ Pro: No stale data
 - └─ Con: Write penalty, unnecessary data cached

Flow:

1. Application write data
2. Write vào database
3. Đồng thời write vào cache
4. Return success

Ưu điểm:

- Cache luôn fresh, không bị stale
- Read performance cao

Nhược điểm:

- Write penalty (phải write 2 nơi)
- Có thể cache data không bao giờ được đọc

Pattern 3: Session Store

- └ Pattern 3: Session Store
 - └ Store user session data (with TTL)
 - └ Makes application stateless
 - └ Use case: User authentication across instances

Flow:

1. User login → Create session
2. Store session data trong ElastiCache (with TTL)
3. Mọi request sau đó authenticate qua cache
4. Application server có thể stateless

Ưu điểm:

- Application stateless → Dễ scale horizontally
- Fast session lookup
- Automatic session expiry với TTL

Use case:

- User authentication
- Shopping cart data
- User preferences

4. REDIS SPECIFIC FEATURES

High Availability

- └ High Availability
 - └ Multi-AZ with automatic failover
 - └ Read replicas for read scaling

- **Multi-AZ deployment** với automatic failover
- Primary node fails → Replica được promote tự động
- **Read Replicas** để scale read performance

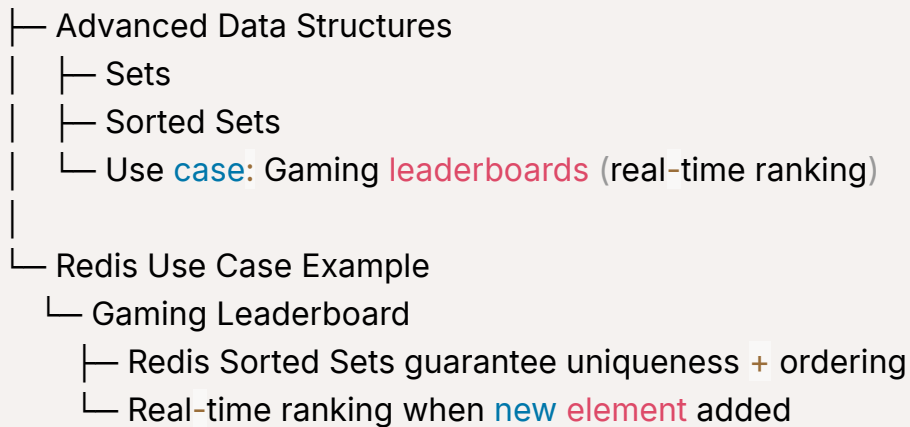
Data Durability & Backup

- └ Data Durability (Persistence)
 - └ AOF (Append-Only File) persistence
- └ Backup & Restore
 - └ Automated backups
 - └ Retention: Up to 35 days
 - └ Manual snapshots

Backup & Restore:

- Y chang bên các RDS hay Aurora
- **Data retention up to 35 days**
- Automated backups và manual snapshots

Advanced Data Structures



Redis Sorted Sets cho Gaming Leaderboard:

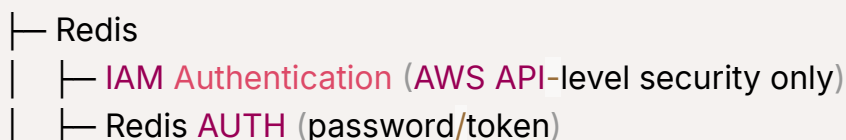
- **Uniqueness:** Mỗi player chỉ có 1 score
- **Ordering:** Tự động sắp xếp theo score
- **Real-time ranking:** Khi thêm score mới, ranking update ngay lập tức
- **Fast queries:**
 - Top N players
 - Player rank
 - Players within score range

Example commands:

```
ZADD leaderboard 1000 "player1" # Add/update score
ZRANGE leaderboard 0 9          # Top 10 players
ZRANK leaderboard "player1"     # Get player rank
```

5. SECURITY

Redis Security



- | └─ SSL encryption in-flight
- | └─ Security Groups for network control

Chi tiết:

- **In-flight encryption:** SSL/TLS support
- **At-rest encryption:** Encryption của data trên disk
- **IAM Authentication:** AWS API-level security
- **Redis AUTH:** Password/token authentication
- **Security Groups:** Network-level firewall

Memcached Security

- └─ Memcached
 - └─ SASL-based authentication (advanced)
 - └─ Security Groups for network control

Chi tiết:

- **SASL authentication:** Advanced authentication mechanism
- **Security Groups:** Network-level firewall
- **Lưu ý:** Memcached có ít security features hơn Redis

KEY TAKEAWAYS

ElastiCache Use Cases

1. Caching Layer:

- Cache database queries
- Giảm tải cho RDS/Aurora
- Improve read performance

2. Session Store:

- User authentication

- Shopping cart
- Stateless applications

3. **Real-time Analytics:**

- Gaming leaderboards
- Rate limiting
- Pub/Sub messaging (Redis)

4. **Distributed Lock:**

- Coordination giữa distributed systems
- Redis SETNX command

Khi nào dùng ElastiCache?

✓ Dùng ElastiCache khi:

- Read-heavy workload cần latency thấp
- Cần cache để giảm tải database
- Cần session store cho stateless app
- Cần real-time leaderboard/ranking

✗ Không dùng ElastiCache khi:

- Primary data store (dùng RDS/Aurora thay thế)
- Cần complex queries và transactions
- Cần ACID guarantees
- Budget constraints (in-memory storage đắt hơn)

Pattern Selection

- **Lazy Loading:** Most common, balance giữa performance và cost
- **Write Through:** Khi cần data consistency cao
- **Session Store:** Khi cần stateless application architecture

- **Combine patterns:** Thường kết hợp Lazy Loading + Write Through cho best results